Spectral State Space Models for Horizontal Connections and Video Modeling

April 3, 2026

Abstract

...

1 Introduction

Visual perception relies on the integration of information across space and time. In the visual cortex, neurons are organized topographically and interact through local horizontal connections, allowing activity to propagate across the visual field while preserving structure. These circuits are thought to underlie key perceptual phenomena such as contour integration and perceptual grouping, where global structure emerges from repeated local interactions.

Computational models have long sought to capture this principle. Architectures based on horizontal recurrent interactions implement local connectivity together with iterative processing, enabling the emergence of long-range dependencies from short-range computations. However, these models are typically formulated as nonlinear recurrent neural networks (RNN), which are inherently sequential and therefore difficult to scale to large inputs or long time horizons. As a result, there is a gap between biologically grounded models of horizontal processing and modern scalable architectures.

State space models (SSMs) have recently emerged as a powerful alternative for sequence modeling, combining linear recurrent dynamics with efficient parallel algorithms. Their favorable scaling properties make them an appealing backbone for modeling long-range dependencies. This raises a natural question: can SSMs serve as a scalable foundation for models of horizontal interactions? Existing extensions of SSMs to high-dimensional data suggest a negative answer. Current approaches either decouple spatial and temporal dynamics or restrict spatial interactions in a way that eliminates true spatio-temporal coupling. More fundamentally, directly incorporating convolutional spatial interactions into SSMs leads to a kernel growth phenomenon that breaks the parallel algorithms underlying their efficiency.

In this work, we resolve this limitation by introducing a spectral formulation of convolutional state space models. By exploiting the fact that convolutional operators are diagonalized in the Fourier domain, we transform *spatio-temporal entanglement* into *decoupled frequency channels*. This representation preserves both spatio-temporal interactions and efficient parallel computation.

We introduce the Spectral Convolutional State Space Model (ConvSSM), a scalable framework that combines the inductive bias of horizontal connectivity with the computational advantages of state space models. This formulation provides a principled backbone for computational models of visual processing, enabling the study of horizontal interactions in regimes that were previously inaccessible to recurrent approaches.

2 The Spectral ConvSSM

The SSM. is a class of recurrent model with linear recurrent dynamics. Their most general formulation is that of continuous-time (Equation (1)), which can be discretised as in Equation (2), with transition matrices depending on the desired step size following Equation (3). This continuous time formulation allows the handling of arbitrary sampling rates for free.

$$\boldsymbol{h}'(t) = \boldsymbol{A}\boldsymbol{h}(t) + \boldsymbol{B}\boldsymbol{x}(t) \quad (1a) \qquad \boldsymbol{h}_t = \overline{\boldsymbol{A}}\boldsymbol{h}_{t-1} + \overline{\boldsymbol{B}}\boldsymbol{x}_t \quad (2a)$$

$$\boldsymbol{y}(t) = \boldsymbol{C}\boldsymbol{h}(t) + \boldsymbol{D}\boldsymbol{x}(t) \quad (1b) \qquad \boldsymbol{y}_t = \boldsymbol{C}\boldsymbol{h}_t + \boldsymbol{D}\boldsymbol{x}_t \quad (2b)$$

Discretization. With zero-order hold (ZOH) and step size Δ,

$$\overline{\boldsymbol{A}} = e^{\Delta \boldsymbol{A}} \qquad \overline{\boldsymbol{B}} = (\Delta \boldsymbol{A})^{-1}(e^{\Delta \boldsymbol{A}} - \boldsymbol{I}) \cdot \Delta \boldsymbol{B}. \tag{3}$$

Proposition 1 (SSM Complexity)**.** *The parallel scan over the SSM has work and time complexity of:*

$$W(\text{Scan}) = O(T), \qquad T_\infty(\text{Scan}) = O(\log T)$$

Diagonal SSMs. In the case of scalar inputs and the convolutional representation, Gu et al. [2022b] show that if the transition matrix $\boldsymbol{A}$ is well conditioned, computing the convolution kernel only requires knowing its spectrum $\sigma(\boldsymbol{A})$. For any vector-valued input SSM, with $\boldsymbol{A} = \boldsymbol{P}\boldsymbol{\Lambda}\boldsymbol{P}^{-1}$ diagonalizable, we can rewrite the SSM as follows:

$$\widetilde{\boldsymbol{A}} = \boldsymbol{\Lambda} \qquad \widetilde{\boldsymbol{B}} = \boldsymbol{P}^{-1}\boldsymbol{B} \qquad \widetilde{\boldsymbol{C}} = \boldsymbol{C}\boldsymbol{P} \tag{4}$$

These two formulations are equivalent in the sense that they produce the same output $\widetilde{\boldsymbol{y}}(t) = \boldsymbol{y}(t)$, with hidden states linearly related by $\widetilde{\boldsymbol{h}}(t) = \boldsymbol{P}^{-1}\boldsymbol{h}(t)$. The diagonal formulation allows to reduce the constant factors in Proposition 1 from $O(N^3)$ to $O(N)$, as shown in Appendix A.5.

3 Convolutional State Space Models

3.1 The ConvSSM

Our goal is to extend state space models to high-dimensional spatio-temporal data such as images and videos. This requires combining *local interactions in space* with *efficient long-range integrations in time*. Convolutions provide a natural inductive bias for spatial locality, while SSMs offer scalable temporal dynamics through linear recurrences. We therefore introduce convolutional state space models (ConvSSMs) as a principled translation of horizontal recurrent circuits into the SSM framework, unifying spatial coupling and temporal dynamics in a single, scalable model. Rewriting the SSM equations (Equation (1)) for kernel convolutions, we get the following spatial ConvSSM:

$$\frac{\partial \boldsymbol{h}}{\partial t}(r,t) = \boldsymbol{K}_h * \boldsymbol{h}(r,t) + \boldsymbol{K}_x * \boldsymbol{x}(r,t) \quad (5a) \qquad \boldsymbol{h}_t = \overline{\boldsymbol{K}}_h * \boldsymbol{h}_{t-1} + \overline{\boldsymbol{K}}_x * \boldsymbol{x}_t \quad (6a)$$

$$\boldsymbol{y}(r,t) = \boldsymbol{K}_y * \boldsymbol{h}(r,t) \quad (5b) \qquad \boldsymbol{y}_t = \boldsymbol{K}_y * \boldsymbol{h}_t \quad (6b)$$

Where r is the spatial position, $\boldsymbol{K}$ are convolution kernels and $*$ is the convolution operator. However, the spatial ConvSSM faces two closely linked challenges as a direct consequence of spatio-temporal coupling. (*Challenge A:*) Left in this format, it is not clear how to compute discrete-time kernels $\overline{\boldsymbol{K}}_h$ and $\overline{\boldsymbol{K}}_x$ from their continuous-time counterparts $\boldsymbol{K}_h$ and $\boldsymbol{K}_x$, or what such kernels would even look like. Specifically, Equation (3) requires matrices and cannot be applied to get the discrete-time kernels $\overline{\boldsymbol{K}}_h$ and $\overline{\boldsymbol{K}}_x$. (*Challenge B:*) parallel algorithms break down when adapted to the spatial ConvSSM (Proposition 2).

Proposition 2 (Spatial ConvSSM Complexity)**.** *Let $(\mathcal{M}, \oplus, e)$ be the monoid corresponding to the spatial convolutional SSM, defined by Equations* (20) *to* (22)*. Let $(a_i)_{i \in [\![0,T-1]\!]} \in \mathcal{M}^{[\![0,T-1]\!]}$. Then, the parallel scan over (a_i) has work and time complexity of:*

$$W(\text{Scan}) = T_\infty(\text{Scan}) = O(T^2)$$

Proof. See Appendix A.3. Key observation: $\oplus$ uses the block convolution operator $\circledast$, which produces kernels of size **exponential in depth**. Therefore, the scan is no longer $O(1)$ per $\oplus$ operation. □

3.2 Spectral ConvSSMs

These are not implementation issues but structural. To circumvent both challenges, we move from a spatial to a spectral representation of the ConvSSM. Spectral representations offer the natural basis we need by

decomposing spatio-temporal entanglement into independent frequency channels. We start by stacking all spatial positions and channels into a single vector, and rewrite the convolutions as matrix-vector products. Indeed, discrete-space convolutions correspond to linear operators with a block-circulant with circulant blocks (BCCB) structure (Equation (7)), leading to an SSM with $d = hwc_{\text{in}}$-dimensional inputs and outputs, and $N = hwc$-dimensional hidden states:

$$\boldsymbol{A} = \text{Circ}(\boldsymbol{K}_h) \qquad \boldsymbol{B} = \text{Circ}(\boldsymbol{K}_x) \qquad \boldsymbol{C} = \text{Circ}(\boldsymbol{K}_y) \tag{7}$$

These matrices are diagonalized by the Fourier transform. We can thus introduce the *Spectral State Space Model*.

Spectral State Space Models. Consider the case where $\boldsymbol{A}$ is block-circulant with blocks of size $c \times c$. This can be rewritten as a 1D convolution over $\frac{N}{c}$ positions and c channels. Let $\boldsymbol{F}_{\frac{N}{c}} \in \mathbb{R}^{\frac{N}{c} \times \frac{N}{c}}$ be the Fourier matrix. Then, $\boldsymbol{A}$ is diagonalized by $\boldsymbol{P} = \boldsymbol{F} \otimes \boldsymbol{I}_c$, where $\otimes$ is the Kronecker product. It can be written as $\boldsymbol{A} = (\boldsymbol{F} \otimes \boldsymbol{I}_c)\boldsymbol{\Lambda}_c(\boldsymbol{F}^{-1} \otimes \boldsymbol{I}_c)$, where $\boldsymbol{\Lambda}_c$ is a block-diagonal matrix with blocks of size $c \times c$. The resulting SSM is what we call a Spectral State Space Model (SSSM).

The core idea of SSSMs is to constrain $\boldsymbol{A}$ not only such that it is structured after some change of basis $\boldsymbol{P}$, but such that $\boldsymbol{P}$ itself is structured, allowing for efficient matrix-matrix and matrix-vector products. This isn't restricted to 1D fourier transforms. For example, this still works when $\boldsymbol{F}_{\frac{N}{c}}$ is replaced by $\boldsymbol{F}_h \otimes \boldsymbol{F}_w$ for 2D convolutions.

$$\widetilde{\boldsymbol{A}} = \boldsymbol{\Lambda}_c \qquad \widetilde{\boldsymbol{B}} = \boldsymbol{P}^{-1}\boldsymbol{B} \qquad \widetilde{\boldsymbol{C}} = \boldsymbol{C}\boldsymbol{P} \tag{8}$$

Diagonal SSSMs. At this point, we have Equation (8) where the change of basis is structured. This means that powering $\widetilde{\boldsymbol{A}}$ costs $O(\frac{N}{c}c^3) = O(Nc^2)$. We go back to the diagonal case where powering $\boldsymbol{A}$ only costs $O(N)$, as this is the bottleneck in parallel algorithms (Appendix A.5). We can write $\widetilde{\boldsymbol{A}} = \boldsymbol{Q}_c\boldsymbol{\Lambda}\boldsymbol{Q}_c^{-1}$, where $\boldsymbol{\Lambda}$ is diagonal and $\boldsymbol{Q}_c$ is a block-diagonal matrix with blocks of size $c \times c$. Essentially, $\boldsymbol{Q}_c$ captures channel mixing withing each frequency block in $\boldsymbol{\Lambda}_c$, and leaves a pure diagonal $\boldsymbol{\Lambda}$. We get the following diagonal SSM:

$$\widehat{\boldsymbol{A}} = \boldsymbol{\Lambda} \qquad \widehat{\boldsymbol{B}} = \boldsymbol{Q}_c^{-1}\boldsymbol{P}^{-1}\boldsymbol{B} \qquad \widehat{\boldsymbol{C}} = \boldsymbol{C}\boldsymbol{P}\boldsymbol{Q}_c \tag{9}$$

The Spectral ConvSSM. In order to combine Equations (7) and (9) we proceed to the following reformulation. **(A)** First, we move from convolution operators to their BCCB matrix counterpart. **(B)** Then, we diagonalize these BCCB matrices through the Fourier transform. We are left with block-diagonals instead of pure diagonals due to channel mixing. **(C)** The final step is therefore to absorb the channel mixing of the recurrent dynamics into the input and output transforms $\boldsymbol{B}$ and $\boldsymbol{C}$. We begin by can rewrite the convolutional SSM as:

$$\boldsymbol{A} = \text{Circ}(\boldsymbol{K}_h) = \boldsymbol{P}\boldsymbol{\Lambda}_c^A\boldsymbol{P}^{-1} = \boldsymbol{P}\boldsymbol{Q}_c\boldsymbol{\Lambda}\boldsymbol{Q}_c^{-1}\boldsymbol{P}^{-1} \tag{10}$$

$$\boldsymbol{B} = \text{Circ}(\boldsymbol{K}_x) = \boldsymbol{P}\boldsymbol{\Lambda}_c^B\boldsymbol{P}_{\text{in}}^{-1} \tag{11}$$

$$\boldsymbol{C} = \text{Circ}(\boldsymbol{K}_y) = \boldsymbol{P}_{\text{in}}\boldsymbol{\Lambda}_c^C\boldsymbol{P}^{-1} \tag{12}$$

with $\boldsymbol{P} = (\boldsymbol{F}_h \otimes \boldsymbol{F}_w) \otimes \boldsymbol{I}_c$, $\boldsymbol{P}_{\text{in}} = (\boldsymbol{F}_h \otimes \boldsymbol{F}_w) \otimes \boldsymbol{I}_{c_{\text{in}}}$. $\text{Circ}(\boldsymbol{K})$ is the convolution operator with kernel $\boldsymbol{K}$, and the $\boldsymbol{\Lambda}_c^{\cdot}$ are block-diagonal matrices with blocks of size $c \times c$, $c \times c_{\text{in}}$ and $c_{\text{in}} \times c$, respectively, corresponding to the Fourier transform of the convolution kernels. This decomposition can therefore be written as a diagonal SSSM, where all the computational load is absorbed by $\boldsymbol{B}$ and $\boldsymbol{C}$:

$$\boxed{\widehat{\boldsymbol{A}} = \boldsymbol{\Lambda}} \tag{13}$$

$$\widehat{\boldsymbol{B}} = \boldsymbol{Q}_c^{-1}\boldsymbol{P}^{-1}\boldsymbol{P}\boldsymbol{\Lambda}_c^B\boldsymbol{P}_{\text{in}}^{-1} = \left(\boldsymbol{Q}_c^{-1}\boldsymbol{\Lambda}_c^B\right)\boldsymbol{P}_{\text{in}}^{-1}\boxed{= \boldsymbol{\Lambda}_c^{B'}\boldsymbol{P}_{\text{in}}^{-1}} \tag{14}$$

$$\widehat{\boldsymbol{C}} = \boldsymbol{P}_{\text{in}}\boldsymbol{\Lambda}_c^C\boldsymbol{P}^{-1}\boldsymbol{P}\boldsymbol{Q}_c = \boldsymbol{P}_{\text{in}}\left(\boldsymbol{\Lambda}_c^C\boldsymbol{Q}_c\right)\boxed{= \boldsymbol{P}_{\text{in}}\boldsymbol{\Lambda}_c^{C'}} \tag{15}$$

This formulation makes it apparent that the tension in *Challenge A* is that the discretized kernels have full spatial support, rendering the spatial convolution representation inapplicable. The Spectral ConvSSM trivializes the discretization process through diagonalization, and bypasses prohibitive convolutions through efficient matrix multiplications. Similarly, *Challenge B* also boils down to kernel size explosion, and is bypassed by the spectral formulation for the same reason. We derive the full complexity analysis in Appendix A and point exactly where the kernel explosion happens for the spatial ConvSSM, and how the spectral formulation avoids it.

Proposition 3 (Spectral ConvSSM Complexity)**.** *Let $(\mathcal{M}, \oplus, e)$ be the monoid corresponding to the spectral convolutional SSM, defined by Equations* (20) *to* (22)*. Let $(a_i)_{i\in[\![0,T-1]\!]} \in \mathcal{M}^{[\![0,T-1]\!]}$. Then, the parallel scan over (a_i) has work and time complexity of:*

$$W(\text{Scan}) = O(T), \qquad T_\infty(\text{Scan}) = O(\log T)$$

Proof. See Appendix A.4. Key observation: the scan is $O(1)$ per $\oplus$ operation. □

3.3 Link with previous work

Previous approaches such as ConvS5 [Smith et al., 2023a] faced these challenges by forfeiting horizontal connections. They restrict the transition operator to act independently at each spatial location, typically using 1×1 convolutions in Equation (5). Equivalently, in the stacked vector representation (Equation (7)), this corresponds to replacing the circulant structure of $\boldsymbol{A}$ with a block diagonal with constant blocks $\boldsymbol{A} = \boldsymbol{I}_{hw} \otimes \boldsymbol{K}$ - where $\boldsymbol{K} \in \mathbb{R}^{c\times c}$.

This constraint removes spatio-temporal coupling from the temporal ODE, i.e., the "Conv" in "ConvS5". In contrast, our approach explicitly models spatio-temporal coupling, disentangling it through the spectral domain while retaining the computational benefits of structured state space models.

3.4 Final Model

[Gregoire]: Find a better section title...

The Core. [Gregoire]: Spectral ConvSSM

Parameterization. [Gregoire]: Rework this paragraph

We parameterize $\boldsymbol{A}$ as a complex-valued vector $\boldsymbol{\Lambda} \in \mathbb{C}^N$. As for $\boldsymbol{B}$ and $\boldsymbol{C}$, we can either parameterize them as kernels in the spatial domain, or as block-diagonal matrices in the spectral domain. The spatial parameterization leads to input and output transforms of $O(cc_{\text{in}}k^2)$ parameters and either $\underbrace{O(hwk^2cc_{\text{in}})}_{\text{spatial convolution}} + \underbrace{O(hw\log(hw)c)}_{\text{FFT}}$ or $\underbrace{O(hwcc_{\text{in}})}_{\text{spectral product}} + \underbrace{O(hw\log(hw)cc_{\text{in}})}_{\text{FFT}}$ complexities depending on the order of operations. The spectral parameterization leads to $O(hwcc_{\text{in}})$ parameters with complexity of $O(hwcc_{\text{in}})$.

Stability. [Gregoire]: eigenvalues of A

Circuits. [Gregoire]: All circuit derivation on the SSM applies to our core. We take Gated Delta Net as state of the art circuit and plug it on top of our core.

4 Scaling Laws

5 Benchmarks

6 Fine Tuning for Video Modeling

7 Interpretability

8 Related Work

State space models (SSMs) have recently emerged as a powerful alternative to attention-based architectures for long-range sequence modeling, combining linear recurrent dynamics with efficient parallel algorithms. Early work on structured state space models builds on the HiPPO framework [Gu et al., 2020], which introduced

principled continuous-time representations of long-range dependencies, and culminates in S4 [Gu et al., 2022a], where carefully parameterized transition matrices together with a convolutional representation and parallel algorithms enable efficient modeling of long sequences. Subsequent work has further simplified and structured SSMs along multiple axes to improve efficiency and scalability. Diagonal parameterizations [Gu et al., 2022b, Gupta and Berant, 2022] reduce computational cost while retaining performance, while extensions from single-input single-output to multi-input multi-output [Smith et al., 2023b] formulations were enabled by efficient parallelization through scan algorithms.

Additional refinements such as linear recurrent units (LRUs) [Orvieto et al., 2023] and gating mechanisms [Mehta et al., 2023] further enhance expressivity and stability. Further circuit derivations and refinements around the core of SSMs include input-dependent selection of transition matrices in Mamba [Gu and Dao, 2023], and the SSM–attention duality in Mamba-2 [Dao and Gu, 2024] through the lens of semiseparable matrices. However, these advances remain fundamentally limited to one-dimensional sequence modeling and do not natively capture spatial structure.

Extending SSMs to high-dimensional visual data such as images and videos requires incorporating dynamics over multiple axes. S4ND [Nguyen et al., 2022] introduces precisely this extension, formalising SSMs evolving over multiple dimensions through separable dynamics. Though successful as a drop-in replacement for state-of-the-art visual processing layers, S4ND's fundamental separation between space and time limits its ability to capture spatio-temporal coupling. Other SSM-based approaches to vision and video modeling have emerged [Zhu et al., 2024, Li et al., 2024a,b].

True spatio-temporal coupling was formulated in Smith et al. [2023a]. Their ConvS5 beats ConvLSTMs and Transformers on long-horizon video modeling. However, ConvS5's convolutional recurrence is limited to 1×1 kernels, thereby forfeiting spatio-temporal coupling of the ConvSSM. This is not a mere design choice, but a fundamental consequence of parallelising over coupled spatio-temporal dynamics. In contrast, our Spectral ConvSSM formulation explicitly models spatio-temporal coupling through the spectral domain while retaining the computational benefits of structured state space models.

9 Conclusion

References

Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning*, 2024.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.

Albert Gu, Tri Dao, Stefano Ermon, Atri Rudra, and Christopher Ré. HiPPO: Recurrent memory with optimal polynomial projections. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

Albert Gu, Karan Goel, and Christopher Ré. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2022a.

Albert Gu, Ankit Gupta, Karan Goel, and Christopher Ré. On the parameterization and initialization of diagonal state space models. In *Advances in Neural Information Processing Systems*, volume 35, 2022b.

Ankit Gupta and Jonathan Berant. Diagonal state spaces are as effective as structured state spaces. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Kunchang Li, Xinhao Li, Yi Wang, Yinan He, Yali Wang, Limin Wang, and Yu Qiao. VideoMamba: State space model for efficient video understanding. In *European Conference on Computer Vision*, 2024a.

Shufan Li, Harkanwar Singh, and Aditya Grover. Mamba-ND: Selective state space modeling for multi-dimensional data. In *European Conference on Computer Vision*, 2024b.

Harsh Mehta, Ankit Gupta, Ashok Cutkosky, and Behnam Neyshabur. Long range language modeling via gated state spaces. In *International Conference on Learning Representations*, 2023.

Eric Nguyen, Karan Goel, Albert Gu, Gordon Downs, Preey Shah, Tri Dao, Stephen Baccus, and Christopher Ré. S4ND: Modeling images and videos as multidimensional signals with state spaces. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Antonio Orvieto, Samuel L. Smith, Albert Gu, Anushan Fernando, Caglar Gulcehre, Razvan Pascanu, and Soham De. Resurrecting recurrent neural networks for long sequences. In *International Conference on Machine Learning*, 2023.

Jimmy T.H. Smith, Shalini De Mello, Jan Kautz, Scott Linderman, and Wonmin Byeon. Convolutional state space models for long-range spatiotemporal modeling. In *Advances in Neural Information Processing Systems*, volume 36, 2023a.

Jimmy T.H. Smith, Andrew Warrington, and Scott Linderman. Simplified state space layers for sequence modeling. In *International Conference on Learning Representations*, 2023b.

Lianghui Zhu, Bencheng Liao, Qian Zhang, Xinlong Wang, Wenyu Liu, and Xinggang Wang. Vision mamba: Efficient visual representation learning with bidirectional state space model. In *International Conference on Machine Learning*, 2024.

A Complexity Proofs

We analyse the complexity of the parallel scan algorithm to show why convolutions in spatial domain make the complexity explode, and how doing convolutions in spectral domain preserves $O(\log T)$ complexity. We consider both the work complexity W, measuring the total amount of atomic operations to perform, and the time complexity under unlimited parallelization T_∞, measuring the longest sequence of atomic operations remaining after optimal parallelization.

We begin by addressing the main issue of the complexity explosion of convolutional SSMs in Appendices A.1 to A.4. Our main result is Proposition 3. We show in Appendix A.4 that we can achieve the same work and time complexity as a regular linear SSM. We then tackle the question of reducing the constant to make it tractable, using standard tricks, adapted to our framework, in Appendix A.5.

A.1 The Parallel Scan

We first describe the parallel scan operation in the most general way. This operation is the core of the SSM parallelization. Let $(a_i)_{i\in I} \in \mathcal{M}^I$ be elements of a monoid $(\mathcal{M}, \oplus, e)$, with $I = [\![0, T-1]\!]$. The parallel scan is an algorithm that computes $(s_i = \sum_{j=0}^{i} a_j)_i \in I$. This algorithm is parallelisable and requires associativity of the $\oplus$ operator and a neutral element e, hence the minimal requirement of a monoid structure. It proceeds in two sweeps mirroring each other. The first sweep proceeds in a tree like structure, with level l of the tree computing $a_k^{l+1} \coloneqq a_{2k}^l \oplus a_{2k+1}^l$, and initialised with $a_k^0 \coloneqq a_k$. Once this upward sweep is over, the downward sweep computes mirror operations, with $b_1^0 \coloneqq e$ and $(b_{2k}^{l+1}, b_{2k+1}^{l+1}) = (b_k^l, b_k^l \oplus a_{2k}^{\log_2(T)-(l+1)})$. The order is crucial here as $\oplus$ is not required to be commutative. It is only required to be associative. In our application for SSM, it is not be commutative. We denote the parallel scan procedure as $\text{Scan}((a_i)_{i\in I})$ or Scan for short. Figure 1 provides a visualisation of the entire procedure.

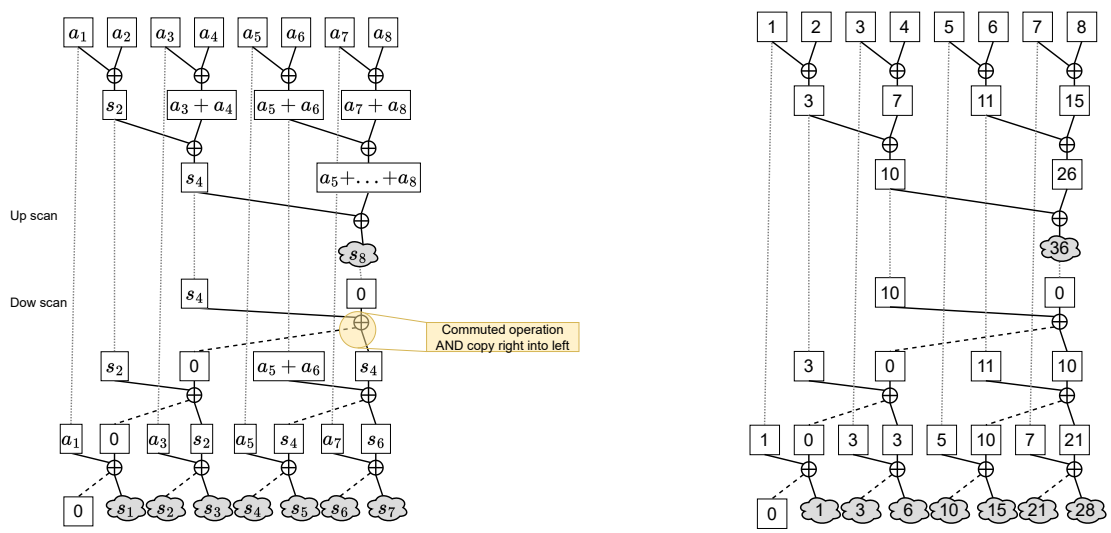

Figure 1: **Left:** parallel scan with a generic monoid. **Right:** illustration on the special case of natural integers.

The procedure described above yields $(\sum_{j=0}^{i-1} a_j)_{i\in I}$, with the cumulative sum being $a_0^{\log_2(T)}$ the intermediate result of the upward sweep. We analyse the work complexity W and the time complexity T_∞ of Scan in terms of floating point operations, where W counts the total number of operations needed and T_∞ counts the time complexity under unlimited parallelism.

Proposition 4 (Parallel Scan Complexity for Integers)**.** *Let $(\mathcal{M}, \oplus, e) \coloneqq (\mathbb{N}, +, 0)$ be the monoid of natural integers equipped with addition. Let $(a_i)_{i\in I} \in \mathbb{N}^I$ be a sequence of integers.*

$$W(\text{Scan}) = 2(T-1) \tag{16}$$
$$T_\infty(\text{Scan}) = 2\log_2 T \tag{17}$$

Proof. At level l of the upward scan, $U_l = T/(2^{l+1})$ $\oplus$ operations are performed, each of them having a complexity of 1. There are $\log_2 T$ levels in total. The same goes on for the downward scan, with $D_l = T/(2^{\log_2(T)-l}) = 2^l$ operations at level l. Therefore, the work complexity of level l is U_l (resp. D_l) while its time complexity is 1 with unlimited parallelism. It follows that

$$W = 2\sum_{l=0}^{\log_2 T-1} 2^l = 2(T-1) \tag{18}$$
$$T_\infty = 2\sum_{l=0}^{\log_2 T-1} 1 = 2\log_2 T \tag{19}$$

□

The key point of this analysis is that the $\oplus$ operation has constant time complexity.

A.2 SSM Complexity

The case of SSMs is a little more complicated than that of real numbers. In SSMs, the recurrence is defined by Equation (2). We take $B = I$ and $d = N$ for simplicity. Thus, the goal is to compute in parallel the hidden states $(h_i = \sum_{j=0}^{i} \overline{A}^{i-j} x_j)_i \in I$, where x_i are vectors of dimension d and $\overline{A}$ is a square matrix. This is almost but not quite what we need for the parallel scan. We define the following monoid:

$$\mathcal{M}^{\text{SSM}} := \left\{a = (A, x) \mid A \in \mathcal{M}_d(\mathbb{R}),\ x \in \mathbb{R}^d\right\} \tag{20}$$
$$(A_a, x_a) \oplus^{\text{SSM}} (A_b, x_b) := (A_b \odot A_a, A_b \cdot x_a + x_b) \tag{21}$$
$$e^{\text{SSM}} := (I_d, 0) \tag{22}$$

With this monoid structure, and the sequence $(a_t)_{t\in I} := ((\overline{A}_t, x_t))_{t\in I} \in \mathcal{M}^I$, the result of the parallel scan is $((A^t, \sum_{j=0}^{t} A^{t-j} x_j))_{t\in I}$. The complexity of the scan is less straightforward, but the complexity of each $\oplus^{\text{SSM}}$ operation depends only on d. It is a constant of T. Therefore, the overall time complexity of the scan is $T_\infty = O(\log T)$ and $W = O(T)$.

Proposition 5 (SSM Complexity)**.** *Let* $(\mathcal{M}, \oplus) := (\mathcal{M}^{\text{SSM}}, \oplus^{\text{SSM}})$ *be the monoid defined by Equations* (20) *to* (22)*. Let* $(a_i)_{i\in I} \in (\mathcal{M}^{\text{SSM}})^I$*.*

$$W(\text{Scan}) = 2(T-1)(d^3 + d^2 + d) \tag{23}$$
$$T_\infty(\text{Scan}) = 2\log_2(T)(d^3 + d^2 + d) \tag{24}$$

Proof. The proof of Proposition 4 applies with the only difference that now, each $\oplus^{\text{SSM}}$ operation has complexity d^3 for the matrix multiplication on the left, d^2 for the matrix-vector multiplication and d for the vector addition on the right. □

A.3 Spatial ConvSSM Complexity

Rewriting the SSM (Equation (2)) to the ConvSSM formulation of Equation (6), we can similarly adapt the monoid structure as :

$$\mathcal{M}^{\text{Conv}_1} \coloneqq \left\{a = (\boldsymbol{K}, x) \mid \exists k_1, k_2, \quad \boldsymbol{K} \in \mathbb{R}^{k_1 \times k_2 \times c \times c},\ x \in \mathbb{R}^{h \times w \times c}\right\} \tag{25}$$

$$(\boldsymbol{K}_a, x_a) \oplus^{\text{Conv}_1} (\boldsymbol{K}_b, x_b) \coloneqq (\boldsymbol{K}_b \circledast \boldsymbol{K}_a, \boldsymbol{K}_b * x_a + x_b) \tag{26}$$

$$e^{\text{Conv}_1} \coloneqq (I, 0) \tag{27}$$

where $\circledast$ is the block convolution between two kernels and $*$ is the convolution between a kernel and an image:

$$\forall \begin{cases} i \in [\![0, h-1]\!] \\ j \in [\![0, w-1]\!] \\ c_{\text{out}} \in [\![1, c]\!] \end{cases}, (\boldsymbol{K} * x)_{i,j,c_{\text{out}}} = \sum_{u=0}^{k_1-1} \sum_{v=0}^{k_2-1} \sum_{c_{\text{in}}=1}^{c} \boldsymbol{K}_{u,v,c_{\text{in}},c_{\text{out}}} x_{(i-u,j-v)\%(h,w),c_{\text{in}}}$$

$$\forall \begin{cases} u \in [\![0, k_1+l_1-2]\!] \\ v \in [\![0, k_2+l_2-2]\!] \\ c_{\text{in}}, c_{\text{out}} \in [\![1, c]\!] \end{cases}, (L \circledast \boldsymbol{K})_{u,v,c_{\text{in}},c_{\text{out}}} = \sum_{p=0}^{l_1-1} \sum_{q=0}^{l_2-1} \sum_{c_k=1}^{c} L_{p,q,c_k,c_{\text{out}}} \boldsymbol{K}_{u-p,v-q,c_{\text{in}},c_k},\ \boldsymbol{K} \text{ being zero-padded}$$

with $L \in \mathbb{R}^{l_1 \times l_2 \times c \times c}$, $\boldsymbol{K} \in \mathbb{R}^{k_1 \times k_2 \times c \times c}$ and $x \in \mathbb{R}^{h \times w \times c}$.

With this structure, the key aspect is that the spatial size of the kernel resulting from $\circledast$ is the sum of the sizes of the input kernels. Therefore, the parallel scan algorithm operates on larger and larger kernel sizes, with sizes growing exponentially with the depth in the tree structure. The complexity of the $\oplus^{\text{Conv}_1}$ operator is thus no more constant as it depends on the depth. The bottleneck is at the very last layer of the upward scan, where kernel size is $O(Tk)$, with k the size of the original kernels $\overline{\boldsymbol{K}}_t$, resulting in a single $*$ operation whose complexity is in $O(T^2)$. This bottleneck is illustrated in Figure 2.

Proposition 6 (Spatial ConvSSM Complexity)**.** *Let* $(\mathcal{M}, \oplus) \coloneqq (\mathcal{M}^{\text{Conv}_1}, \oplus^{\text{Conv}_1})$ *be the monoid defined by Equations* (20) *to* (22)*. Let* $(a_i)_{i \in I} \in (\mathcal{M}^{\text{Conv}_1})^I$*.*

$$W = O(T^2) \tag{28}$$

$$T_\infty = O(W) \tag{29}$$

Proof. The proof of Proposition 4 applies with the major difference that in the upward scan, at level l, each $\oplus^{\text{Conv}_1}$ operation has complexity $(k + (2^l - 1)(k-1))^2 \times c^3 = O((2^l)^2 \times k^2 \times c^3)$ for the kernel-kernel convolution on the left, $O(hw \times (2^l)^2 k^2 \times c^2)$ for the kernel-image convolution and hwc for the image addition on the right. The same goes on for the downward scan.

Thus, Equations (18) and (19) become :

$$W = O(2 \sum_{l=0}^{\log_2(T)-1} 2^{\log_2(T)-(l+1)} \times \left[(2^l)^2 k^2 \times c^3 + hw \times (2^l)^2 k^2 \times c^2\right]) \tag{30}$$

$$= O(T^2 \times k^2 \times c^3) + O(T^2 \times hw \times k^2 \times c^2) \tag{31}$$

$$T_\infty = O(2 \sum_{l=0}^{\log_2(T)-1} \left[(2^l)^2 k^2 \times c^3 + hw \times (2^l)^2 k^2 \times c^2\right]) \tag{32}$$

$$= O(W) \tag{33}$$

□

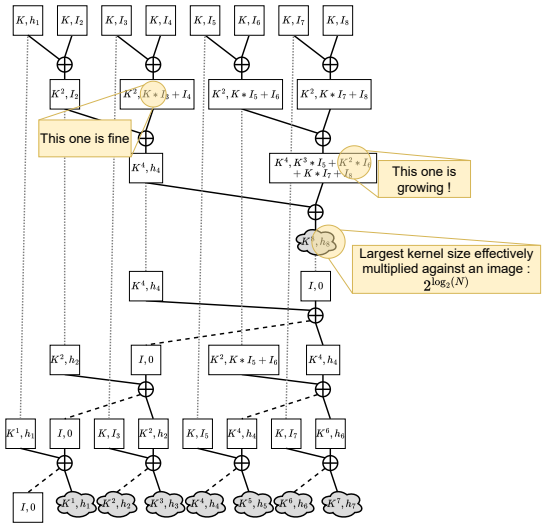

Figure 2: Parallel scan applied to the monoid structure defined for spatial convolution SSM. Highlighted is the core of the issue : growing kernels lead to a bottleneck where a single operation has $O(T^2)$ complexity.

Moreover, as mentioned in the main text, the discretization step of the SSM leads to kernels with full spatial support, meaning that the initialization of the scan is $(a_t)_{t\in I} = ((\overline{\boldsymbol{K}}_t, x_t))_{t\in I}$ with $\overline{\boldsymbol{K}}_t$ of size $h\times w$. Thus, even the first step of the scan is intractable.

A.4 Spectral ConvSSM Complexity

To dispell the curse of growing kernels we move to the spectral domain and the ConvSSM formulation of Equation (13). In this domain, our monoid becomes:

$$\mathcal{M}^{\text{Conv}_2} \coloneqq \left\{ a = (\boldsymbol{\Lambda}_c, x) \;\middle|\; \exists (\boldsymbol{K}, x') \in \mathcal{M}^{\text{Conv}_1}, \begin{cases} \boldsymbol{\Lambda}_c = P^{-1}\text{Circ}(\boldsymbol{K}_{\text{padded}})P \in \mathbb{C}^{hwc\times hwc} \\ x = P^{-1}x' \in \mathbb{C}^{hwc} \end{cases} \right\} \tag{34}$$

$$(\boldsymbol{\Lambda}_c^a, x_a) \oplus^{\text{Conv}_2} (\boldsymbol{\Lambda}_c^b, x_b) \coloneqq (\boldsymbol{\Lambda}_c^b \odot \boldsymbol{\Lambda}_c^a, \boldsymbol{\Lambda}_c^b \cdot x_a + x_b) \tag{35}$$

$$e \coloneqq (I, 0) \tag{36}$$

with $\odot$ the element-wise matrix multiplication and $\cdot$ the element-wise matrix-vector multiplication. With this operator, the kernel grows no more, the cost is again a constant of time and depth, depending only on h, w and c. We are thus back to a time complexity for the parallel scan of $O(\log T)$.

Proposition 7 (Spectral ConvSSM Complexity)**.** *Let* $(\mathcal{M}, +) \coloneqq (\mathcal{M}^{\text{Conv}_2}, \oplus^{\text{Conv}_2})$ *be the monoid defined by Equations* (34) *to* (36)*. Let* $(a_i)_{i\in I} \in (\mathcal{M}^{\text{Conv}_2})^I$*.*

$$W = O(T) \tag{37}$$

$$T_\infty = O(\log_2 T) \tag{38}$$

Proof. The proof of Proposition 4 applies with the only difference that now, each $\oplus^{\text{Conv}_2}$ operation has complexity hwc^3 for the kernel-kernel multiplication on the left, hwc^2 for the kernel-image multiplication and hwc for the image addition on the right. This gives :

$$W = O(T \times hwc^3) \tag{39}$$

$$T_\infty = O(\log_2 T \times hwc^3) \tag{40}$$

□

A.5 Taking down the constant factors

Now that we have solved the main issue of complexity explosion with time, we tackle the problem of the constant factor. As mentioned in Proposition 1, as is, the work complexity is $O(T)$ with a constant that is $O(d^3)$ due to matrix multiplications. The diagonal SSM (Equation (4)) reduces this constant from $O(d^3)$ to $O(d)$. The complexity gain comes from the change of basis being absorbed outside of the recurrence such that the recurrence can operate on diagonalized matrices.

In the case of the Spectral SSM (Equation (8)), the $O(c^3)$ is even more prohibitive as it is repeated over each pixel position, with the total constant being $O(hwc^3)$ in the spectral domain (Equations (39) and (40)). Hence the diagonal spectral SSM (Equation (9)), which reduces the constant to $O(hwc)$. Our ConvSSM formulation (Equation (13)) exploits this diagonal spectral SSM and couples it with convolutional input and output transforms in order to make the whole model tractable.